Design of a Multi-Agent RAG Architecture to Support the AREU (118) Operations Center

Andrea De Tomasi

April 2026

Abstract

The integration of large language models (LLM) in critical sectors is often hindered by structural limitations such as *hallucinations* and the static nature of knowledge. We describe the design and implementation of a multi-agent Retrieval-Augmented Generation (RAG) architecture intended to support *Agenzia regionale Emergenza Urgenza* (AREU) operators in the rapid consultation of complex procedures and medical protocols.

The system is orchestrated using LangGraph in conjunction with DecAgents, a proprietary library of Ottante SRL (part of the Beta 80 group) based on LangGraph and LangChain. By leveraging the core functionalities and networking capabilities of DecAgents—a library already hosted in several cloud-based production environments—the architecture overcomes the linearity of traditional pipelines by implementing a computational graph in which specialized autonomous agents collaborate on query decomposition, semantic retrieval, and result validation. To maximize retrieval accuracy, a query expansion strategy based on hypothetical queries was implemented, along with a reranking system using a sigmoid function to minimize informational noise. The entire infrastructure is hosted on Amazon Bedrock, ensuring high standards of security and confidentiality when handling sensitive data from the Agency.

Tests conducted with the direct involvement of AREU personnel demonstrate that integrating reasoning mechanisms at each node of the graph significantly enhances the system's transparency and reliability. Although the evaluated models (Amazon Nova-2 Lite and Claude Sonnet) achieved similar satisfaction scores, performance analysis identified latency as a critical factor for emergency operations, with document filtering emerging as the primary bottleneck. In conclusion, this work illustrates how a multi-agent approach can transform corporate documentation into an intelligent and secure dialogue, providing tangible decision-making support in high-stress operational scenarios.

Table of Contents

Abstract . . . 2

1 Introduction . . . 4

1.1 The Role of AREU and the Management of 118 . . . 4

1.2 The Need for an Intelligent Assistant . . . 4

1.3 Operational Impact and Future Vision . . . 5

2 State of the Art: Retrieval-Augmented Generation (RAG) Systems . . . 6

2.1 Genesis of the RAG Paradigm . . . 6

2.2 Architecture and Components of a RAG System . . . 6

2.3 Evolution of RAG Paradigms . . . 8

2.4 Agentic Frontiers, Evaluations, and Future Challenges . . . 10

2.5 Conclusion . . . 11

3 Methods . . . 12

3.1 System Architecture . . . 12

3.1.1 Selection of Database and Models . . . 12

3.1.2 Orchestration Frameworks: LangGraph and DecAgents . . . 12

3.1.3 Pipeline Description . . . 13

3.2 Dataset and Preprocessing . . . 15

3.3 Retrieval with Reasoning . . . 16

3.4 Generation and Validation . . . 17

4 Results . . . 19

5 Discussion . . . 22

Bibliography . . . 24

Appendix . . . 25

1 Introduction

The integration of Artificial Intelligence in critical sectors represents one of the most ambitious challenges of the modern era. While large language models (LLMs) have demonstrated remarkable capabilities in structuring and leveraging natural language semantic rules, they exhibit significant structural limitations, including a tendency to produce incorrect information (so-called *hallucinations*) and possessing static knowledge, limited to the period of their training [Lew+20]. In domains where information accuracy can determine the success or failure of an intervention, these limitations are unacceptable.

1.1 The Role of AREU and the Management of 118

This work is situated at the core of the Lombard rescue system: the Regional Emergency Agency (AREU), the organization responsible for coordinating the Unique Emergency Number 118. Working in an emergency center entails managing extremely high-pressure situations, where both the speed at which information can be retrieved and shared, and the reliability of such piece of information are critical. Operators must have immediate access to a vast body of knowledge, including operational instructions, internal procedures, and complex medical protocols—often contained in dense documents that are potentially of difficult consultation quickly during an emergency call.

1.2 The Need for an Intelligent Assistant

Our goal is to develop a multi-agent architecture based on the Retrieval-Augmented Generation (RAG) paradigm, specifically designed to support the center's personnel. Unlike a conventional AI system, this architecture does not rely solely on its internal "memory"; instead, it functions as an expert librarian: upon receiving a query, it consults an external memory composed of AREU's official documents, extracts relevant information, and processes it to provide a clear and verifiable response. This approach ensures that every suggestion provided by the agent is grounded in reliable and up-to-date sources, significantly reducing the risk of errors. In particular, the proposed system makes the RAG pipeline fully auditable. Furthermore, the decision to leverage advanced infrastructures such as Amazon Bedrock is both technical and strategic, enabling the management of sensitive data while maintaining maximum control and confidentiality—an essential requirement when handling rescue procedures and internal protocols.

1.3 Operational Impact and Future Vision

The proposed RAG-methodology did not emerge in isolation but it is the result of a process involving the direct participation of AREU operators. The operators validated the relevance of the responses and the usability of the API interface (query, retrieval, sources validation). With this effort, we aim to provide a decision-support tool capable of serving a variety of professional roles, ranging from field rescuers to healthcare personnel, and even logistics managers.

In summary, the introduction of this technology aims to transform the way information is accessed during emergencies, moving from a manual and potentially slow search process to swift information retrieval via natural language interaction with corporate documentation, thereby supporting faster, better-informed, and safer decision-making for the operators, and ultimately for the public.

2 State of the Art: Retrieval-Augmented Generation (RAG) Systems

2.1 Genesis of the RAG Paradigm

In recent years, Large Language Models (LLMs) have demonstrated remarkable capabilities in learning both factual and semantic knowledge from vast amounts of textual data. However, these models exhibit significant structural limitations, including difficulties with continuous updating, limited interpretability, and a tendency to produce incorrect or unverifiable information—a phenomenon referred to as "hallucination". These limitations stem from the parametric nature of the knowledge encoded in the models: it is embedded in the weights as the result of a large-scale optimization process and remains largely static.

From a mathematical perspective, this knowledge corresponds to a point (or region) of minimum in the loss function within the parameter space. Consequently, modifying it in a targeted manner requires a new training phase or re-optimization, as it is not possible to adjust individual pieces of content locally without affecting the global balance of the learned parameters.

To address these challenges, the Retrieval-Augmented Generation (RAG) paradigm was introduced by Lewis et al. in 2020 [Lew+20]. In this framework, the model's parametric knowledge is complemented by a non-parametric external memory, composed of collections of pre-indexed documents.

2.2 Architecture and Components of a RAG System

A RAG system is based on three fundamental components:

- retrieval from external sources
- knowledge integration
- text generation

Access to external knowledge sources is achieved through semantic retrieval techniques, which go beyond simple lexical matching of keywords by representing both queries and documents as vectors in a continuous semantic space (embeddings). In this vector space, the

similarity between texts is computed based on the distance or angle between their respective vectors (e.g., using cosine similarity), enabling the retrieval of relevant documents even when they do not share exact terms with the query but convey the same conceptual content. This approach allows the model to dynamically incorporate updated or specialized information without modifying its internal parameters.

The component typically employed for this phase in a RAG system is the neural retriever, which consists of one or more neural encoders (often based on Transformer architectures) trained to map queries and documents into dense vectors within a shared semantic space. In the most common bi-encoder configuration, one encoder processes the query while another processes the documents (sometimes sharing weights). Each text is thus mapped to a vector $f(x) \in \mathbb{R}^d$, where the dimension d is predetermined.

The Transformer architecture, introduced by Vaswani et al. (2017) [Vas+17], relies entirely on attention mechanisms, fully eliminating recurrences and convolutions. Its core component, self-attention, enables each element of a sequence to dynamically weigh all other elements, modeling long-range dependencies in parallel. Given a set of queries Q, keys K, and values V, the scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^{\mathbb{T}}}{\sqrt{d_k}}\right)V$$

where d_k is the size of the keys. The normalization factor $\frac{1}{\sqrt{d_k}}$ stabilizes the gradient for large vector dimensions.

Instead of applying a single attention function with keys, values, and queries of size d_{model}, it is advantageous to project linearly the queries, keys, and values h times through different learned linear projections, respectively towards spaces of size d_k, d_k, and d_v. Multi-head attention enables the model to simultaneously attend to information from different representation subspaces at various positions. In contrast, with a single attention head, the averaging operation tends to limit this capability:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

$$\text{where} \quad \text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right)$$

and W_i^Q, W_i^K, W_i^V and W^O are learned parametric matrices.

Thanks to this architecture, the Transformer enables fully parallel processing of sequences and efficient modeling of global dependencies, forming the foundation of modern large language models.

The loss function serves as the mathematical criterion guiding the training of the retriever. It is typically contrastive: given a set of relevant (positive) and non-relevant (negative) query-document pairs, the loss is designed to maximize the similarity between vectors of positive pairs while minimizing that of negative pairs. Formally, the encoder parameters are optimized so that the geometric structure of the vector space reflects a notion of semantic relevance. In this way, the learned function induces a metric in the embedding space that approximates the concept of relevance relative to the query.

The most relevant documents identified by the retriever are then provided as context to the generative model, which leverages them to produce the answer. This integration of semantic retrieval and text generation allows the model's parametric knowledge to be enriched with updatable external information, enhancing performance in knowledge-intensive tasks.

This paradigm has been shown to improve the factual accuracy, transparency, and updatability of LLM-based systems, while reducing the need for continuous retraining. Moreover, the ability to trace information sources during generation represents a key factor for trust and verifiability of responses [Gao+23].

2.3 Evolution of RAG Paradigms

Recent literature highlights a progressive evolution of RAG architectures, marked by increasing modularity and a closer integration between retrieval and generation. Early implementations followed a linear pipeline, in which a query was transformed into an embedding, used to retrieve documents, and then fed to the generative model [Gao+23].

In this initial phase, retrieval mechanisms were largely derived from traditional Information Retrieval: lexical approaches such as BM25 [Rob+94] were gradually complemented and eventually replaced by dense neural models based on vector representations shared between queries and documents. The adoption of semantic embeddings enhanced both the robustness and generalization capabilities of these systems.

As the paradigm matured, research introduced modular improvements aimed at optimizing the quality of context provided to the generator. These include strategies for document chunking (segmenting into coherent informative units), query expansion, dynamic context selection, re-ranking, and semantic filtering. Such techniques reflect the recognition that the

notion of relevance in generative systems differs from that in traditional retrieval systems [Che+25].

A further evolution concerns the integration of retrieval and reasoning. Modern architectures incorporate multi-hop approaches, in which the system performs multiple iterations of retrieval and reasoning, as well as query decomposition strategies. This enables the handling of complex tasks that require distributed inferences across multiple documents or heterogeneous sources.

From this perspective, a key aspect is the alignment between the retriever and the generator. Zhou & Chen (2025) propose an end-to-end optimization of the retriever based on in-context relevance, demonstrating that relevance for generative contexts differs from that in traditional retrieval systems [ZC25]. Their approach highlights how the efficiency of retrieval can significantly impact overall performance, even when using smaller generative models.

Simultaneously, recent studies underscore the importance of managing the context window of LLMs. Techniques such as semantic compression, adaptive document selection, and noise control aim to optimize the use of available context. In particular, Cuconasu et al. (2024) show that appropriate filtering and weighting of retrieved documents reduces noise and improves model effectiveness [Cuc+24], while Cheng et al. (2025) provide a systematic overview of the most recent strategies for efficiently combining and selecting documents [Che+25].

A counterintuitive finding in the literature concerns the role of noise in the retrieval process. Cuconasu et al. (2024) demonstrate that the controlled introduction of partially irrelevant documents can actually improve the accuracy of the generative model, suggesting that a more diverse context facilitates the inferential process [Cuc+24]. A possible theoretical explanation is that moderately heterogeneous information acts as a form of contextual regularization: including documents that do not strictly overlap with the query may reduce the risk of overfitting the model to partial or redundant evidence, promoting a more robust latent representation. Moreover, a diversified context can expand the semantic space explored by the model during generation, increasing the likelihood of activating relevant but not immediately obvious conceptual connections.

Another hypothesis is that controlled noise improves attention calibration: the need to discriminate between strongly and weakly relevant signals may lead the model to weigh the available information more selectively, reinforcing the integration and comparison mechanisms across sources. In this view, the observed positive effect does not arise from the

noise itself but from its role in stimulating selection and inferential composition within the context window. This finding challenges traditional retrieval evaluation metrics and opens new perspectives for the design of RAG systems.

Finally, there is growing interest in integrating multimodal systems, structured databases, and knowledge graphs, extending the role of retrieval beyond simple access to textual documents and positioning RAG systems as hybrid infrastructures for knowledge access and composition.

2.4 Agentic Frontiers, Evaluations, and Future Challenges

Another research direction concerns computational efficiency and scalability. The adoption of distributed architectures, incremental indexing, and semantic caching enables the deployment of RAG systems in real-world contexts characterized by latency and cost constraints. Furthermore, integration with agentic systems and automated workflows represents a rapidly growing area. By agentic system, we refer to an architecture in which the language model is not limited to generating a response from a single prompt, but functions as an agent capable of planning, making intermediate decisions, and interacting with external tools (e.g., search engines, databases, APIs, or computation modules).

The cutting edge of research is shifting toward Multi-Agent Systems (MAS). In this paradigm, task complexity is not handled by a single "generalist agent," but distributed across a network of specialized agents that collaborate. This decomposition allows assigning distinct roles to different logical entities, such as:

- Research Agent: specializes in semantic query expansion (hypothetical queries) and precise retrieval of chunks.
- Critical or Validator Agent: evaluates the relevance of sources and the coherence of responses, triggering correction cycles if quality criteria are not met.
- Synthesizer Agent: focuses on final text generation, ensuring that outputs are anchored to sources and free from hallucinations.

This shift toward a multi-agent approach provides a theoretical justification for the use of computational graph architectures, where each node represents not a simple function but an agent with its own domain of expertise. The MAS approach enhances both modularity and interpretability: since each agent must provide reasoning for its actions, it becomes possible to trace precisely where errors may occur in the reasoning chain.

Despite progress in defining and implementing efficient and comprehensive RAG systems, several challenges remain, including:

- alignment between retrieval and generation
- efficient knowledge selection
- interpretability of systems
- realistic performance evaluation
- management of obsolete or contradictory information
- definition of realistic benchmarks and appropriate evaluation metrics

2.5 Conclusion

The RAG paradigm has evolved from a simple document retrieval system into a complex, modular, and knowledge-oriented ecosystem. The dynamic integration of external knowledge enhances the reliability, factual accuracy, and transparency of language models, making these systems increasingly suitable for real-world applications in specialized domains.

Future directions include multimodal systems, autonomous agents, adaptive retrieval, integration with knowledge graphs, and advanced reasoning strategies. These innovations have the potential to enable the development of more robust, interpretable, and updatable intelligent systems.

3 Methods

3.1 System Architecture

3.1.1 Selection of Database and Models

For the implementation of the system, a targeted selection was made regarding both the database used for storing information and the models employed in the retrieval and generation phases. In particular, PostgreSQL was chosen as the database management system, enhanced with pgvector, an extension that enables efficient storage and search of vector embeddings. This solution allows the integration of vector search capabilities directly within a relational database, simplifying the system's infrastructure and facilitating the management of documents and their embeddings used in the retrieval process.

Regarding artificial intelligence models, the Amazon Bedrock service was employed, providing access to a variety of foundational models through a unified interface. Using this platform enables the integration of models for both embedding and generation, maintaining a flexible and scalable architecture while facilitating experimentation and the replacement of models without substantial modifications to the system infrastructure. Furthermore, leveraging Amazon Bedrock allows for controlled access to models via the Amazon Web Services infrastructure, ensuring greater data governance and helping to preserve the confidentiality of documents used by the system — a particularly important consideration in the context of this work.

3.1.2 Orchestration Frameworks: LangGraph and DecAgents

For the implementation and orchestration of the RAG system workflow, LangGraph was selected — a library designed for building applications based on Large Language Models (LLMs) using graph-based computational structures. This choice stems from the need to define a structured processing pipeline that enables dynamic integrations across different phases, while providing modularity and flexibility in execution, thereby ensuring the robustness required in a critical context such as AREU. LangGraph extends the paradigm of linear pipelines commonly used in LLM frameworks, allowing for the definition of more complex execution flows characterized by conditional transitions and iterative cycles. The implementation, however, does not rely solely on LangGraph; it also integrates the core functionalities and networking layers of DecAgents. DecAgents is a proprietary library developed by Ottante SRL, a company within the Beta 80 group, designed to facilitate the

creation of multi-agent systems with a focus on modularity, scalability, and production readiness.

Unlike traditional sequential architectures, in which operations follow a static pipeline, LangGraph represents the process as a directed graph composed of nodes and edges. In this framework, each node represents a computational unit — such as a retrieval, reasoning, or generation module — while the edges define the flow of data and the conditions for transitioning between the different system phases. In practice, each node functions as an autonomous agent executing a specific role.

Another advantage offered by LangGraph is the ability to assign different models to each node based on the type of task to be performed. This means that, depending on the complexity and criticality of the operation, it is possible to use more specialized and high-performance models or lighter, less expensive versions. For example, simple data processing or filtering tasks can be handled by more economical models, while complex response generation or advanced reasoning can leverage more powerful models. This flexibility allows for a balanced optimization between computational cost and performance quality, dynamically allocating resources according to the specific needs of each node. Moreover, it facilitates the integration of new models or technological updates without the need to redesign the entire workflow, preserving the modularity and scalability of the architecture.

Processing is further guided by a shared graph state, a data structure containing relevant information accumulated during execution. This state is passed between agents and progressively updated, allowing the system to track the current context of a request and the intermediate results generated by the various workflow components.

The adoption of a graph-based architecture has enabled the implementation of more sophisticated reasoning strategies, including iterative cycles between retrieval and generation, quality checks on produced responses, and fallback mechanisms when retrieved information is insufficient. Simultaneously, this structure ensures high modularity: each graph node encapsulates a specific system functionality, allowing individual components — such as the retriever or generative model — to be modified or replaced without affecting the overall workflow. These features enhance system flexibility, supporting both dynamic adaptation during execution and experimentation with alternative architectural configurations.

3.1.3 Pipeline Description

The workflow of the RAG architecture is illustrated in Figure 1, which depicts all phases of the process. Upon receiving a query, the agent [`rag_or_not`] determines whether to

terminate the conversation [`end_conversation`], generate a direct answer [`direct_answer`] without retrieving documents, or proceed with retrieval. This strategy avoids unnecessary transitions to the retrieval phase for questions that do not require document consultation, thereby optimizing both computational costs and response times.

In the retrieval scenario, another agent [`generate_hyq`] generates two questions semantically related to the original query (hypothetical queries), embeds all three queries, and computes the average of the three embeddings to retrieve the most relevant documents from the database. The retrieval agent enriches each retrieved chunk with a relevance score and an explanatory reasoning, and then orders the chunks based on their scores [`retrieve_chunks_with_reasoning`]. The retrieved documents are then provided as context to the generative agent [`gen_rag_answer`], which produces the final response.

The subsequent step involves validating the generated response [`validate_answer`], comparing it with the original query to assess the coherence and correctness of the provided information. If the outcome is negative, the system may iterate the retrieval process, updating the graph state with the newly obtained information and generating additional queries to refine the search, or terminate the conversation if further attempts are unlikely to significantly improve the response (e.g., due to insufficient documentation) [`fallback_message`].

When the response is deemed satisfactory, an agent generates a TL;DR [`add_sources_tldr`] and then the system returns a contextualized answer to the end user, grounded in external sources, thereby enhancing the reliability and relevance of the information compared to a standalone generative model.

All elements generated during the process — queries, retrieved documents, produced responses, and the time taken by each node — are stored in the graph state, allowing the system to maintain a complete trace of the interaction. This information can be used in subsequent iterations or for post-hoc analyses of system performance. Furthermore, every step is accompanied by reasoning, a textual explanation describing the decisions made by the system at each phase, thereby improving transparency and interpretability of the process.

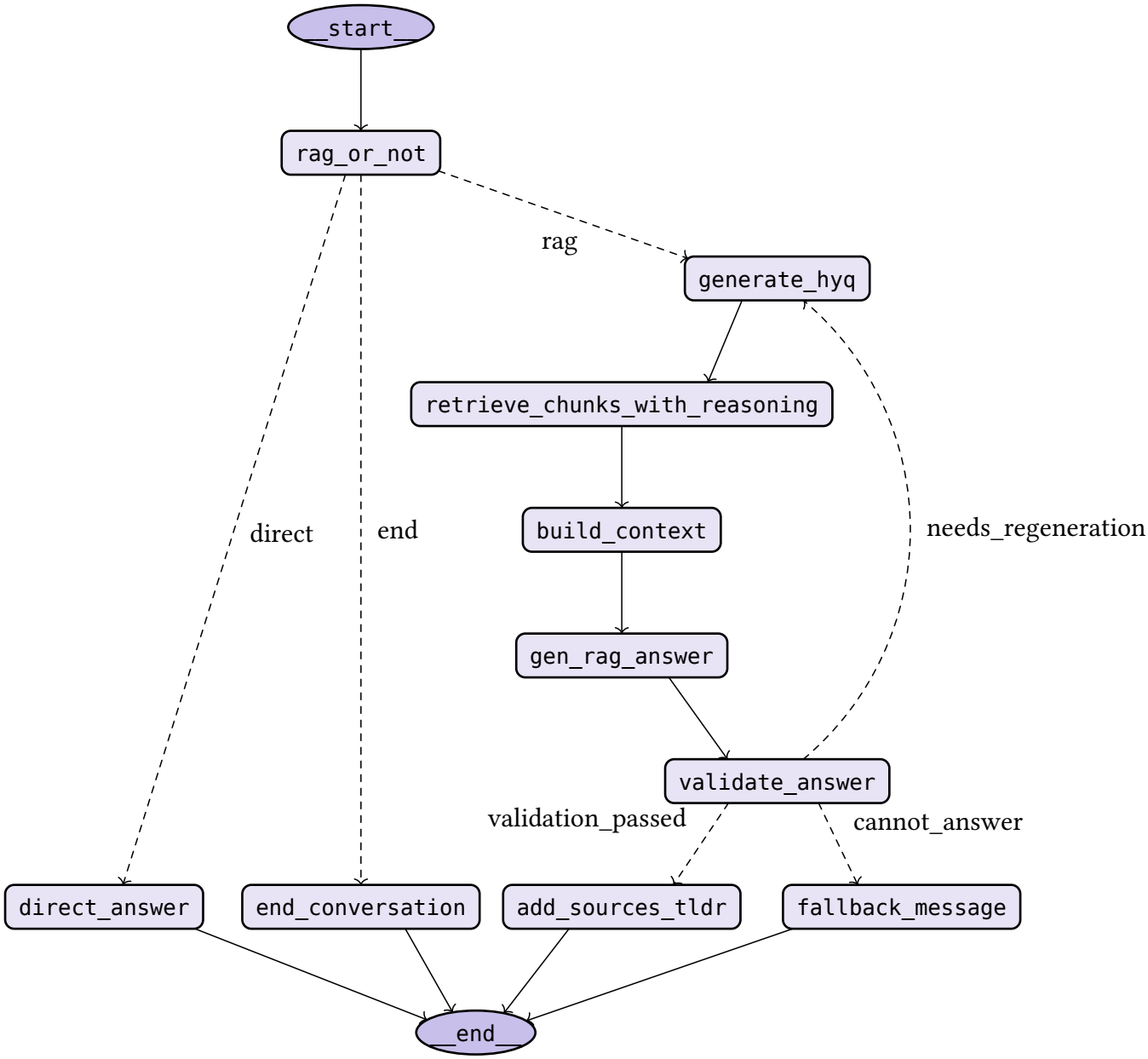

Figure 1: RAG agent workflow

3.2 Dataset and Preprocessing

The documents used for retrieval were collected from both public and private sources and include operational instructions, internal procedures, and other relevant materials for emergency management. The preprocessing pipeline involved data cleaning, text normalization, and segmentation into coherent informative units (chunking). Each sufficiently long chunk is accompanied by a summary and three hypothetical queries — automatically generated questions whose answers are contained within the chunk and represent different informational aspects — to improve semantic coverage during retrieval.

It has been observed that including hypothetical queries alongside the textual content of the chunk significantly enhances retrieval performance.

All chunks have been further enriched with keywords derived from the general context of the document, ensuring that relevant information, even when distributed across different sections of the text, is preserved. This approach guarantees that crucial contextual knowledge is not lost, facilitating accurate and comprehensive retrieval.

The database was implemented using PostgreSQL with the pgvector extension, enabling efficient storage and indexing of vector embeddings associated with the documents. The embeddings were generated using Titan Text Embeddings V2, an embedding model available on Amazon Bedrock, and stored in the database together with document metadata, such as titles and the distinction between public and private documents.

3.3 Retrieval with Reasoning

During the retrieval phase, the system generates two hypothetical queries based on the user's question, transforms them into embeddings, and computes the average of the three embeddings to retrieve the most relevant documents from the database. By considering documents closest to the semantic mean of the three related queries, the system more accurately identifies the chunks that are truly useful for generating the response. Subsequently, the content of the retrieved documents is evaluated to determine the relevance and pertinence of the information with respect to the original query. This evaluation is performed by a model that assigns each chunk a score between 0 and 1, representing the probability of relevance, along with a textual explanation justifying the assessment.

After this step, documents are ranked by relevance and filtered to retain only the most pertinent ones using a sigmoid function centered at the average score with a steepness of 8. This approach reduces noise and improves the quality of the context provided to the generative model. The raw relevance scores are first transformed via the sigmoid function before filtering. Consequently, only documents with a sigmoid value greater than 0.65 are retained, while those with a value ≤ 0.65 are discarded, as illustrated in Figure 2. This re-ranking and filtering strategy allows non-relevant chunks to be discarded without risking the loss of useful information.

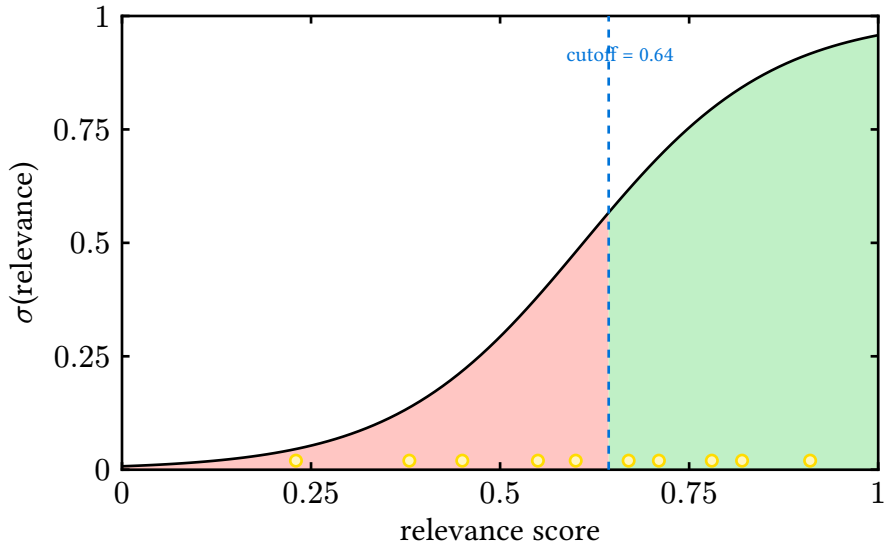

Figure 2: An example sigmoide function used during re-ranking to better separate informative and non-informative chunks

Finally, the selected documents are provided as context to the generative model, retaining information about their relevance to the original query. This allows the agent to integrate the information more effectively during the response generation process.

3.4 Generation and Validation

The generative model leverages the retrieved documents along with their relevance scores, the original query, and the corresponding hypothetical queries to produce a contextualized, source-based response. Response generation is guided by a Jinja2 prompt, designed to encourage the model to integrate information coherently and provide explanations for the decisions made during generation. The use of Jinja2 also enables dynamic prompt structuring, facilitating the insertion and management of variables—such as documents and queries—in a modular, reusable, and maintainable manner. The response includes citations to the documents used, a summary of the content, and an explanation of the inferential process followed by the model to arrive at the final answer.

The validation phase directs the system's flow, determining whether to iterate retrieval or terminate the conversation. In this step, the agent provides a textual explanation describing the rationale behind its decision, enhancing the transparency of the process and allowing the user to understand why the response was deemed satisfactory or not. Specifically, the model assigns a satisfaction score between 0 and 1, calculated according to predefined criteria; a response is considered acceptable when the score is ≥ 0.8. If the score is < 0.8, the system evaluates whether to iterate the retrieval process, updating the graph state with the

newly obtained information and generating additional queries to refine the search, or to terminate the conversation if further attempts are unlikely to significantly improve the response (e.g., due to insufficient documentation). A complete example of the system's output, including the retrieved chunks, relevance scores, and validation reasoning, is provided in the Appendix

4 Results

Preliminary tests were conducted to evaluate the effectiveness of the implemented RAG system, focusing on semantic retrieval quality, response generation quality, and search times. The results indicate that the system is capable of retrieving relevant documents even in the presence of complex or ambiguous queries, thanks to the use of semantic embeddings and the generation of hypothetical queries. The responses produced are contextualized and grounded in reliable sources, with a significant reduction in search times compared to a purely manual approach.

The most impactful factor in retrieval quality was the inclusion of hypothetical queries: in most cases, adding the appropriate question was sufficient to correctly retrieve the most relevant chunk. Additionally, a well-structured user input further improves retrieval performance. Regarding response generation, particular attention was given to prompt engineering, as it emerged that the prompt is a critical determinant of output quality. Given the same informative context and a well-defined query, the structure, clarity, and instructions contained in the prompt significantly influence the coherence, completeness, and reliability of the generated response. Moreover, a clear and precise question enables the model to better identify which information to include, reducing ambiguity and improving the relevance of the output.

AREU operators were directly involved in testing, providing qualitative feedback on the relevance of responses and the usability of the system. To automate the evaluation process and establish a benchmark for future developments, a dataset of questions accompanied by the expected content for their respective answers was constructed. Preliminary results suggest that RAG systems hold significant potential in emergency contexts, where the speed and reliability of information are critical.

To evaluate the impact of model quality and optimize resource usage, only the model in the node responsible for response generation was modified. In the other nodes, the same model was retained, as less complex tasks do not require the use of more powerful and costly models. The three generative models compared were Amazon Nova 2 Lite, Claude Sonnet 4.6, and Mistral Pixtral Large. As shown in Figure 3 A, Sonnet achieved the best overall performance, standing out with the highest number of perfect responses, while Pixtral demonstrated the weakest performance. However, the differences between Sonnet and Nova were less pronounced. This suggests that retrieval quality and prompt structure can have a

more decisive impact on overall performance than the specific choice of generative model, at least within the range of models tested. These results highlight the importance of optimizing retrieval components and prompt engineering to maximize the effectiveness of RAG systems, rather than focusing exclusively on the selection of the generative model.

Regarding response times, Figure 3 panel *B* shows a significant difference in latency between the three models, with Amazon Nova 2 Lite returning responses considerably faster than Claude Sonnet 4.6 and Mistral Pixtral Large. This result underscores the importance of considering not only the quality of generated responses but also the computational efficiency of the models, especially in contexts such as AREU, where timely access to information is critical.

A more detailed analysis of execution times across the other nodes of the graph (Figure 3 *C*) reveals that the re-ranking phase constitutes the primary bottleneck, with execution times significantly higher than those observed in the other phases. This finding emphasizes the need to optimize the evaluation and selection of retrieved documents, as it represents a critical factor for the overall efficiency of the system. Improvements in this phase could lead to substantial reductions in response times, making the system more suitable for real-world scenarios where rapid access to information is essential.

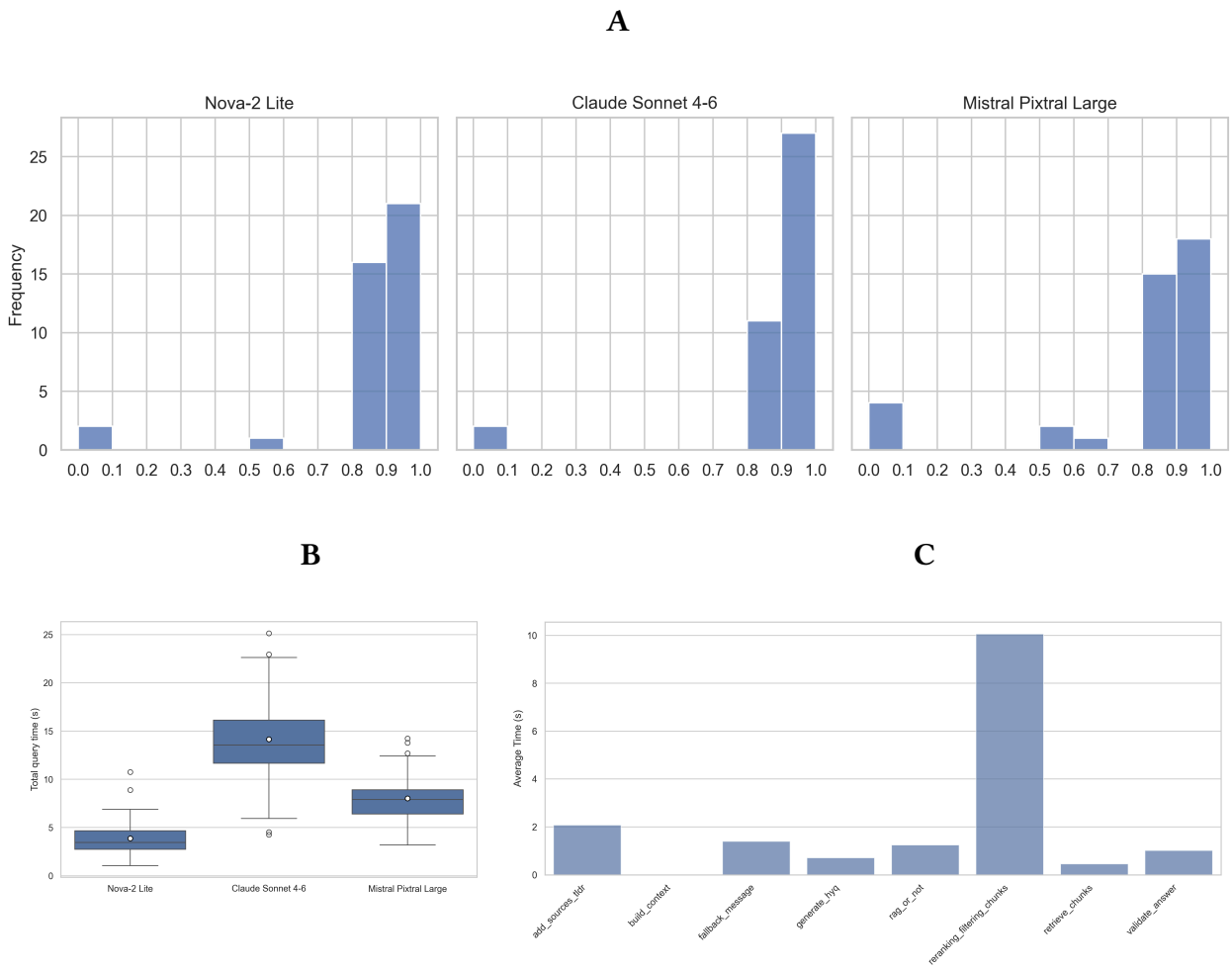

Figure 3: Scores and times comparison.

*Panel **A** shows the score distribution for each model. We can observe that Sonnet is the best overall model.*

*Panel **B** shows the distribution of query times per model as a boxplot. Nova-2-Lite achieves the lowest latency, consistent with its smaller model size.*

*Panel **C** shows the average execution time per pipeline node. The bottleneck is the re-ranking phase, which is significantly more time-consuming than the other phases, suggesting that optimizing this step could lead to substantial improvements in overall system performance.*

5 Discussion

The implemented RAG system has proven effective in retrieving relevant information and generating contextualized responses, thereby providing a viable solution for streamline access to informative content via a well-defined, auditable process capable of providing guarantees about the reliability and relevance of the retrieved information in a pointwise manner. However, several limitations have emerged, particularly the need for further optimization to handle more complex and dynamic scenarios. Notably, the documents used for retrieval were not originally designed for processing within a RAG framework, and therefore exhibit structures that are not optimal for both retrieval and generation tasks.

Moreover, the presence of redundant information (i.e., repeated concepts across sections or documents), obsolete content (outdated directives), and unstructured elements (such as images and tables) hindered, in some occasions, the system's ability to identify and integrate the most relevant information. This highlights the importance of a more targeted data curation and organization process to maximize the effectiveness of RAG systems. Designing documents specifically for RAG usage would reduce the number of chunks passed to the re-ranking phase, thereby improving overall performance, as the system would require less time both to process and to reason over the content.

Among the strengths of the system, the modularity provided by the DecAgents and LangGraph framework stands out. This multi-agent approach, supported by Ottante SRL's production-ready networking library, allows individual components to be updated or replaced without affecting the overall workflow. Additionally, the ability to dynamically integrate external knowledge enhances the factuality and transparency of generated responses. The adoption of a graph-based architecture has also enabled the implementation of more advanced reasoning strategies and the maintenance of a complete trace of interactions, facilitating both iterative refinement and post-hoc analysis of system performance.

Future developments include the integration with existing healthcare information systems, with the aim of further improving the relevance and practical utility of responses for AREU operators. Another promising direction involves the exploration of multimodal approaches, incorporating non-textual sources such as images and structured data to enrich the context available to the generative model. Finally, further research will focus on the systematic

comparison of different embedding and generation models, as well as the exploration of hierarchical ranking strategies to optimize document selection during retrieval.

In conclusion, RAG systems demonstrate significant value in emergency contexts, where speed and reliability of information are critical. By grounding responses in verified documentation, such systems enhance operational efficiency and support more informed decision-making. Their flexibility and adaptability make them suitable for a wide range of professional roles—from field rescuers to healthcare personnel and logistics managers—ultimately contributing to faster, safer, and more effective interventions.

Bibliography

[Lew+20] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[Vas+17] A. Vaswani *et al.*, "Attention Is All You Need," *arXiv preprint arXiv:1706.03762*, 2017, [Online]. Available: https://arxiv.org/abs/1706.03762

[Gao+23] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023, [Online]. Available: https://arxiv.org/abs/2312.10997

[Rob+94] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, "Okapi at TREC-3," *Proceedings of The Third Text REtrieval Conference (TREC 1994)*, 1994, [Online]. Available: https://www.researchgate.net/publication/221037764_Okapi_at_TREC-3

[Che+25] M. Cheng *et al.*, "A Survey on Knowledge-Oriented Retrieval-Augmented Generation," *arXiv preprint arXiv:2503.10677*, 2025, [Online]. Available: https://arxiv.org/abs/2503.10677

[ZC25] J. Zhou and L. Chen, "OpenRAG: Optimizing Retrieval-Augmented Generation End-to-End via In-Context Retrieval Learning," *arXiv preprint arXiv:2503.08398*, 2025, [Online]. Available: https://arxiv.org/abs/2503.08398

[Cuc+24] F. Cuconasu *et al.*, "The Power of Noise: Redefining Retrieval for Retrieval-Augmented Generation," *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024, [Online]. Available: https://arxiv.org/abs/2401.14887

Appendix

The following JSON excerpt illustrates the structure of the evaluation dataset used to assess the system's performance. Each entry encodes a single test question alongside its expected answer, the response generated by the system, the reasoning trace produced during retrieval, the reference documents identified as relevant, the retrieved chunks with their relevance scores, per-node timing breakdowns, and the automated evaluation outcome. The dataset consists of 40 questions derived from AREU operational procedures and was processed using the Nova-2-Lite model configuration. Only a representative sample is reproduced here for brevity.


```
{
  "run_info": {
      "timestamp": "2026-04-02 12:48:55 CEST",
      "agent_config": "eu.amazon.nova-2-lite-v1:0",
      "language": "Italiano",
      "num_questions": 40
  },
  "questions": [
      {
          "id_question": "1",
          "question": "L'operatore tecnico che processa richiesta di soccorso
può declassare in autonomia il codice evento da rosso a giallo?",
          "expected_answer": "Sì - L'operatore tecnico può declassare in
autonomia un codice da rosso a giallo motivando la scelta nelle note
strutturate",
          "answer": "Sì, l'operatore tecnico che processa la richiesta di
soccorso può declassare in autonomia il codice evento da rosso a giallo, a
condizione che venga esplicitata la motivazione nel campo Note. Questa
possibilità è regolamentata specificamente dalle indicazioni contenute nel
documento operativo, che stabilisce che non ci sono limitazioni al
declassamento del codice di gravità da rosso a giallo, a differenza di altri
declassamenti che richiedono una rivalutazione sanitaria.",
          "reasoning": "L'utente chiede se l'operatore tecnico che gestisce le
richieste di soccorso può declassare autonomamente il codice evento da rosso a
giallo. Il contesto fornisce informazioni chiare al riguardo. Il documento IOP
12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf
stabilisce che non ci sono limitazioni al declassamento del codice di gravità
da rosso a giallo, purché venga esplicitata la motivazione nel campo 'Note'.
```

Questo documento è il più rilevante e fornisce una risposta diretta alla domanda dell'utente. Gli altri documenti forniscono contesto aggiuntivo sui codici di gravità e sui ruoli, ma non cambiano la conclusione principale.",
          "tldr": "Puoi declassare un codice evento da rosso a giallo in autonomia, basta che scrivi la motivazione nel campo Note. Questo è permesso dal documento operativo e non serve una rivalutazione sanitaria, a differenza di altri declassamenti.",
          "reference_documents": [
              "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
              "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
              "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
              "DOC 109 Attività consolle flotta e sanitaria Rev1 del 14-07-2015.pdf"
          ],
          "retrieved_chunks": [
              {
                  "chunk_id": "0d3ab50c-302f-4d90-b8f6-78e84e812421_20",
                  "doc_id": 6,
                  "filename": "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
                  "chunk_reasoning": "Questo frammento è direttamente pertinente alla domanda poiché stabilisce chiaramente che l'operatore tecnico può declassare il codice da rosso a giallo senza necessità di rivalutazione sanitaria, a patto di motivare la decisione nel campo 'Note'.",
                  "chunk_relevance": 1.0,
                  "above_threshold": true
              },
              {
                  "chunk_id": "415f1739-51cb-4835-ba2a-f1ffceba0477_24",
                  "doc_id": 84,
                  "filename": "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
                  "chunk_reasoning": "Questo frammento è rilevante perché descrive i codici di gravità degli eventi e menziona che l'operatore sanitario può modificare il codice dopo una rivalutazione, fornendo un contesto utile per capire le possibilità di declassamento.",
                  "chunk_relevance": 0.8,
                  "above_threshold": true
              },

```
            {
                "chunk_id": "e79b3cc3-39ee-4adb-82e6-6367e0081479_24",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento ripete le informazioni
sui codici di gravità e menziona che l'operatore sanitario può modificare il
codice dopo una rivalutazione, fornendo ulteriore contesto ma non risolvendo
direttamente la domanda sull'autonomia dell'operatore tecnico.",
                "chunk_relevance": 0.8,
                "above_threshold": true
            },
            {
                "chunk_id": "0b51feec-ea88-40a6-8f6f-f16cb8274fdd_23",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento descrive i codici
colore e il ruolo dell'operatore tecnico receiver nel definire il codice di
gravità, supportando il contesto della domanda ma non fornendo informazioni
specifiche sul declassamento autonomo da rosso a giallo.",
                "chunk_relevance": 0.7,
                "above_threshold": true
            },
            {
                "chunk_id": "1da9db4b-1b49-49a7-9648-268cef0b7ce6_28",
                "doc_id": 75,
                "filename": "DOC 109 Attività consolle flotta e sanitaria
Rev1 del 14-07-2015.pdf",
                "chunk_reasoning": "Questo frammento indica che la consolle
sanitaria può ridefinire il codice di gravità in qualsiasi momento, ma non
specifica se l'operatore tecnico può farlo autonomamente, quindi è meno
direttamente rilevante della chunk 0d3ab50c-302f-4d90-b8f6-78e84e812421_20.",
                "chunk_relevance": 0.6,
                "above_threshold": true
            },
            {
                "chunk_id": "695b709a-1417-4fc5-9051-6d9881dcbce6_26",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento spiega come l'operatore
```

dispatcher può modificare i codici di invio del mezzo, ma non affronta direttamente la possibilità di declassamento del codice evento da parte dell'operatore tecnico, quindi è meno rilevante.",
"chunk_relevance": 0.5,
"above_threshold": false
},
{
"chunk_id": "60d89164-2741-46fa-b5bf-536677cc87cd_26",
"doc_id": 84,
"filename": "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
"chunk_reasoning": "Questo frammento descrive le azioni dell'operatore dispatcher in caso di codici evento bianchi o verde/giallo, ma non affronta il declassamento da rosso a giallo da parte dell'operatore tecnico, quindi ha una rilevanza moderata.",
"chunk_relevance": 0.5,
"above_threshold": false
},
{
"chunk_id": "9a5c9153-f577-487d-89e5-36bc9aa814d9_63",
"doc_id": 6,
"filename": "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
"chunk_reasoning": "Questo frammento descrive le procedure per la revoca di una richiesta di soccorso e non è direttamente rilevante per la domanda sul declassamento del codice evento, ma fornisce contesto aggiuntivo sulle operazioni dell'operatore tecnico.",
"chunk_relevance": 0.4,
"above_threshold": false
},
{
"chunk_id": "553f7297-0d5d-4ebe-87eb-b995476691ed_28",
"doc_id": 108,
"filename": "DOC 161 Riscontro sanitario SOREU Rev1 del 08-01-2026.pdf",
"chunk_reasoning": "Questo frammento spiega che il codice di primo riscontro può essere variato dal personale sanitario in funzione di aspetti logistici, ma non si riferisce all'operatore tecnico o al declassamento da rosso a giallo, quindi ha una bassa rilevanza.",
"chunk_relevance": 0.4,
"above_threshold": false
},

```
            {
                "chunk_id": "93546983-ca96-4290-bf96-e77ddea3eeb2_27",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento si concentra sui codici
di riscontro in posto e non è direttamente rilevante per la domanda sul
declassamento del codice evento da parte dell'operatore tecnico, quindi ha una
bassa rilevanza.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "4ef1cedf-ad0c-4c0f-912d-aee7a2b6e1dc_36",
                "doc_id": 108,
                "filename": "DOC 161 Riscontro sanitario SOREU Rev1 del
08-01-2026.pdf",
                "chunk_reasoning": "Questo frammento riguarda i pazienti con
patologie tempo-dipendenti e i codici gialli/rossi associati, ma non è
direttamente rilevante per la domanda sul declassamento autonomo del codice
evento da parte dell'operatore tecnico.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "529007cc-7f9c-4f22-8521-fcb86934c155_13",
                "doc_id": 115,
                "filename": "DOC 170 Rivalutazione codici non urgenti Rev0
del 31-05-2024.pdf",
                "chunk_reasoning": "Questo frammento descrive le azioni da
intraprendere da parte dell'infermiere in caso di richiesta di invio di un MSB
e modifica del codice di gravità, ma non si riferisce specificamente al
declassamento da rosso a giallo da parte dell'operatore tecnico, quindi ha una
bassa rilevanza.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "a6e4539d-88c2-4d7f-9f74-b1aff9ede70c_17",
                "doc_id": 115,
                "filename": "DOC 170 Rivalutazione codici non urgenti Rev0
del 31-05-2024.pdf",
```

"chunk_reasoning": "Questo frammento tratta la gestione della revoca del paziente e non è direttamente rilevante per la domanda sul declassamento del codice evento, quindi ha una bassa rilevanza.",
"chunk_relevance": 0.3,
"above_threshold": false
},
{
"chunk_id": "ecb34f68-c440-490b-9297-3a570c43b9d7_31",
"doc_id": 108,
"filename": "DOC 161 Riscontro sanitario SOREU Rev1 del 08-01-2026.pdf",
"chunk_reasoning": "Questo frammento definisce i significati dei codici giallo, rosso e nero per i pazienti, ma non fornisce informazioni sulle procedure di declassamento da parte dell'operatore tecnico, quindi ha una rilevanza molto bassa.",
"chunk_relevance": 0.2,
"above_threshold": false
},
{
"chunk_id": "055023cc-06f8-4d6c-8043-d8f563080040_11",
"doc_id": 192,
"filename": "PRO 21 Gestione regionale mezzi di soccorso Rev6 del 16-01-2024.pdf",
"chunk_reasoning": "Questo frammento descrive la modalità di selezione del mezzo condiviso e non è rilevante per la domanda sul declassamento del codice evento, quindi ha una rilevanza molto bassa.",
"chunk_relevance": 0.2,
"above_threshold": false
},
{
"chunk_id": "674d05d3-2766-4c8d-8917-1bb984f23214_14",
"doc_id": 193,
"filename": "PRO 22 Gestione maxiemergenze Rev5 del 14-07-2025.pdf",
"chunk_reasoning": "Questo frammento riguarda la gestione delle maxiemergenze e il ruolo del Referente Tecnico di Turno, ma non affronta il declassamento del codice evento da parte dell'operatore tecnico, quindi ha una rilevanza molto bassa.",
"chunk_relevance": 0.1,
"above_threshold": false
}
],

```
            "thread_id": "areu_batch_1_20260402_124855",
            "timings": {
                "node_times": {
                    "rag_or_not": 1.574,
                    "generate_hyq": 0.742,
                    "retrieve_chunks": 0.63,
                    "reranking_filtering_chunks": 10.594,
                    "build_context": 0.0,
                    "gen_rag_answer": 2.873,
                    "validate_answer": 0.994,
                    "add_sources_tldr": 1.798
                },
                "rag_invoke_time_s": 19.283,
                "check_time_s": 1.27,
                "total_time_s": 20.553
            },
            "evaluation": {
                "check_passed": true,
                "check_correspondence_score": 0.9,
                "check_missing_points": "none",
                "check_reasoning": "La risposta generata copre tutti i punti
chiave della risposta attesa: conferma che l'operatore tecnico può declassare
il codice da rosso a giallo in autonomia e l'obbligo di motivare la scelta
nelle note. L'informazione aggiuntiva sul documento operativo e sulle
limitazioni per altri declassamenti è corretta e non contraddice la risposta
attesa, quindi non influisce negativamente sulla corrispondenza."
            }
        },
        ...
    ]
}
```